

Dettagli di Build & Deploy

- **Prerequisiti:**

- Clonare il repository dal link GitHub fornito.
- Installare Docker
- Installare Python ed eseguire il comando:
\$ pip install grpcio grpcio-tools google-api-python-client
- Inizializzare il database creando le apposite tabelle (Vedi paragrafo successivo)
- Installare minikube
- Avviare il cluster con il seguente comando
\$ minikube start
- Per semplicità, consigliamo di creare il seguente alias:
\$ alias kubectl="minikube kubectl --"

- **Build e Deploy:**

- Eseguire il seguente comando dalla directory principale del progetto:
\$ kubectl create -f k8s-specifications/
- Essendo il database inizialmente privo di tabelle, il data collector non riuscirà ad eseguire, riavviandosi di continuo, quindi occorre inizializzare il database:
 - Avviare l'interfaccia di Kubernetes con il seguente comando:
\$ minikube dashboard
 - Dall'interfaccia di Kubernetes, andare su:
Pods → db → terminal
 - Eseguire il comando:
mysql -u andrea -p
 - Inserire la password "password"
 - Eseguire il comando:
use hw1
 - Copiare e incollare tutto il contenuto presente nel file "queries.sql" e premere ENTER
 - A questo punto il data collector potrà avviarsi correttamente

- **Testing**

- Essendo che Kubernetes esegue dentro un container docker (creato da minikube) non è possibile accedere alle nodePort dei servizi che devono comunicare con "client" e "prometheus_monitor", ovvero "server" e "prometheus". Pertanto è necessario esporre separatamente tali servizi.
 - Test del client
 - Esporre il server eseguendo il comando:
\$ minikube service server
 - Avviare il client con il seguente comando:
\$ python client.py <porta_specificata_da_minikube>
 - Test di prometheus_monitor
 - Esporre prometheus eseguendo il comando:
\$ minikube service prometheus

- Avviare prometheus_monitor con il seguente comando
\$ python prometheus_monitor.py
<porta_specificata_da_minikube> [-s <nome_servizio>] [-m
<nome_metrica>]